
title: "Algoritmos na Prática" subtitle: "Regressão, Árvores, K-Means e PCA — Da Intuição ao Código de Produção" author: "Nexus Affil'IA'te MMN_IA" collection: "Curso Universo IA — Coletânea Técnica Vol. 03" version: "1.0" date: "2026-06" classification: "Acadêmico · Técnico · Hands-On · PhD-Level"

Capa

"Teoria sem código é filosofia. Código sem teoria é hacking. Os dois juntos são engenharia." — Nexus Affil'IA'te MMN_IA

Sumário

1. **Prólogo — O Algoritmo Não Importa. O Problema Importa.**
2. **Capítulo 1 — Setup do Ambiente Profissional**
3. **Capítulo 2 — Pipeline de Dados: Do Raw ao Model-Ready**
4. **Capítulo 3 — Análise Exploratória (EDA)**
5. **Capítulo 4 — Feature Engineering**
6. **Capítulo 5 — Regressão Linear com NumPy (Do Zero)**
7. **Capítulo 6 — Regressão Linear com Scikit-Learn**
8. **Capítulo 7 — Regressão Logística — Detecção de Spam**
9. **Capítulo 8 — Árvores de Decisão — Risco de Crédito**
10. **Capítulo 9 — Random Forest — A Força do Ensemble**
11. **Capítulo 10 — Gradient Boosting (XGBoost, LightGBM)**
12. **Capítulo 11 — KNN — Sistema de Recomendação**
13. **Capítulo 12 — K-Means — Segmentação de Clientes**
14. **Capítulo 13 — PCA — Compressão de Imagens**
15. **Capítulo 14 — DBSCAN — Detecção de Fraude**
16. **Capítulo 15 — Isolation Forest — Outliers em Séries Temporais**
17. **Capítulo 16 — SVM — Classificação de Texto**
18. **Capítulo 17 — Naive Bayes — Filtro de Spam (Clássico)**
19. **Capítulo 18 — Validação Cruzada e Hyperparameter Tuning**
20. **Capítulo 19 — Pipelines e MLflow**
21. **Capítulo 20 — Deploy: Do Notebook à API**

- 22. Capítulo 21 — Monitoramento em Produção (Drift)
- 23. Capítulo 22 — MLOps: O Ciclo de Vida Completo
- 24. Capítulo 23 — Algoritmos para Time Series
- 25. Capítulo 24 — Algoritmos para NLP Clássico
- 26. Capítulo 25 — Quando NÃO Usar ML
- 27. Glossário
- 28. Referências

Prólogo — O Algoritmo Não Importa. O Problema Importa.

É tentador cair na armadilha de **otimizar o algoritmo** quando o verdadeiro gargalo está em outro lugar:

- **Dados ruins** (mais comum que se imagina).
- **Problema mal formulado** (métrica errada).
- **Distribuição diferente em produção.**
- **Falta de feature engineering.**

Este volume é **hands-on**. Cada capítulo traz: - Conceito teórico - Código Python executável - Caso de uso real - Armadilhas comuns

Vou usar datasets públicos (Iris, Boston Housing, MNIST, Titanic) e casos práticos do **ecossistema Nexus Affil'IA'te**.

Capítulo 1 — Setup do Ambiente Profissional

1.1 Python + Anaconda/Mamba



```

conda create --name ml-proj python=3.11
conda activate ml-proj
pip install torch torchvision torchaudio
pip install numpy pandas scikit-learn matplotlib seaborn
pip install xgboost lightgbm catboost
pip install mlflow wandb

```

1.2 Hardware Recomendado

- **Treino básico:** 16GB RAM, CPU multi-core.
- **Deep Learning:** GPU NVIDIA com 12GB+ VRAM (RTX 3060+).
- **Produção:** Multi-GPU, Kubernetes, GPUs A100/H100.

1.3 Estrutura de Projeto

```

.
├── data
│   ├── raw          # Dados originais (read-only)
│   ├── processed    # Dados processados
│   └── external     # Dados de terceiros
├── notebooks        # Jupyter notebooks
├── scripts           # Scripts de automação
├── features          # Features engineering
├── models            # Modelos treinados
├── tests             # Unit tests
├── config            # Arquivos de configuração
├── mlflow             # MLflow artifacts
├── models_deploy     # Modelos para produção
├── docs              # Documentação
├── requirements.txt  # Dependências

```

1.4 Boas Práticas

- **Reprodutibilidade:** `np.random.seed(42)`, `random_state` em tudo.
- **Versionamento:** Git + DVC para dados.
- **Logging:** MLflow, Weights & Biases.

- **Documentação:** Docstrings em tudo, README claro.
- **Testing:** pytest, data validation com Great Expectations.

Capítulo 2 — Pipeline de Dados: Do Raw ao Model-Ready

2.1 Etapas Fundamentais

```
Raw Data → Ingestion → Cleaning → Transformation → Feature Engineering → Storage
```

2.2 Profiling com Pandas Profiling

```
import pandas as pd
from pandas_profiling import ProfileGenerator

# Carrega o dataset
df = pd.read_csv('dataset.csv')

# Gera o perfil
profile = ProfileGenerator(df)
profile.to_html_report_path('report.html')
```

Gera relatório com: tipos, missing, distribuição, correlações, alertas.

2.3 Tratamento de Missing Values

```
# Detecta missing values
df.isnull().sum()

# Remove linhas com qualquer NaN
df.dropna(inplace=True)

# Imputação simples
df['idade'].fillna(df['idade'].mean(), inplace=True)
df['categoria'].fillna(df['categoria'].mode()[0], inplace=True)

# Imputação avançada com KNN
from sklearn.impute import KNNImputer
imputer = KNNImputer()
df[['idade', 'categoria']] = imputer.fit_transform(df[['idade', 'categoria']])
```

Imputação avançada: - **KNN Imputer:** Usa vizinhos para imputar. - **Iterative Imputer (MICE):** Modela cada feature com missing como função das outras.

2.4 Detecção de Outliers

```
from sklearn.preprocessing import StandardScaler, OutlierRemoval
X = data[['price', 'area']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
scores = zscore(X_scaled['price'])
outliers = data[abs(scores) > 3]
```

2.5 Encoding de Variáveis Categóricas

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
encoder = OneHotEncoder(sparse=False)
encoder.fit(data[['category']])
encoded = encoder.transform(data[['category']])
data_encoded = pd.DataFrame(encoded, columns=encoder.get_feature_names_out())
data_encoded['target'] = data['target']
data_encoded = data_encoded[['category', 'target']]
data_encoded['category'] = data_encoded['category'].astype('category')
data_encoded['category'] = data_encoded['category'].cat.add_categories('new_category', inplace=True)
data_encoded['category'] = data_encoded['category'].cat.remove_unused_categories()
```

2.6 Scaling

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data[['price', 'area']])
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data[['price', 'area']])
scaler = RobustScaler()
data_scaled = scaler.fit_transform(data[['price', 'area']])
```

Quando usar: - **StandardScaler:** Regressão, SVM, KNN, PCA. - **MinMaxScaler:** Redes neurais, quando há limites conhecidos. -

RobustScaler: Dados com outliers. - **Não escalar:** Árvores, Gradient Boosting (já são invariantes a escala).

Capítulo 3 — Análise Exploratória (EDA)

3.1 Os 5 Mandamentos do EDA

1. **Sempre olhe os dados crus** — `df.head()` , `df.sample(10)` .
2. **Entenda as distribuições** — Histogramas, KDE, boxplots.
3. **Procure correlações** — Heatmap, pairplot.
4. **Analise por segmento** — Groupby + agregação.
5. **Questione outliers** — Investigar antes de remover.

3.2 Visualizações Essenciais



3.3 Estatísticas-Chave

Category	Count
1. General Information	15
2. Project Details	20
3. Financials and Budget	18
4. Timeline and Milestones	25
5. Stakeholder Engagement	12

3.4 Quando Suspeitar

- **Distribuição bimodal:** Pode haver dois grupos misturados.
- **Skew extremo:** Variável precisa de transformação (log, sqrt).
- **Valores constantes:** Feature inútil.
- **Alta correlação (>0.95):** Multicolinearidade.

Capítulo 4 — Feature Engineering

4.1 A Ciência Mais Importante

"Better features beat better algorithms." — Pedro Domingos

4.2 Transformações Numéricas

Country	0-14	15-64	65+
USA	75	100	25
Canada	45	65	15
Mexico	35	55	10

4.3 Interações

```
oil_price_per_kil_l = oil_price_l * oil_distance_l
```

4.4 Encoding de Datas

```
oil_date_l = pd.to_datetime(oil_date_l)
oil_year_l = oil_date_l.dt.year
oil_month_l = oil_date_l.dt.month
oil_day_of_week_l = oil_date_l.dt.dayofweek
oil_weekend_l = oil_day_of_week_l.isin([5, 6]) * astype(int)
oil_day_since_l = pd.Timestamp.now().dt.date - oil_date_l.dt.date
```

4.5 Text Features (Básico)

```
oil_text_length_l = oil_text_l.str.len()
oil_num_words_l = oil_text_l.str.split().str.len()
oil_has_question_l = oil_text_l.str.contains('?', astype=bool)
```

4.6 Feature Selection

```
oil_features_to_remove = []
oil_df_with_target = oil_df[oil_df['target'].abs() > 0].sort_values(ascending=False)

# Embedded user feature importance to model
oil_df_train = oil_df[oil_df['target'].abs() > 0].sample(frac=1, random_state=42)
oil_model = RandomForestClassifier()
oil_model.fit(oil_df_train, oil_df_train['target'])

oil_model_feature_importances = oil_model.feature_importances_
oil_top_features = oil_model_feature_importances.argsort()[::-1]

# Wrapper: Recursive Feature Elimination (RFE)
oil_df_train = oil_df[oil_df['target'].abs() > 0].sample(frac=1, random_state=42)
oil_rfe = RFE(estimator=oil_model, n_features_to_select=10)
oil_selected = oil_df_train[oil_rfe.transform(oil_df_train)]
```


Capítulo 5 — Regressão Linear com NumPy (Do Zero)

5.1 O Algoritmo

Code Snippet	Frequency
<code>import numpy</code>	1
<code>ax=plt.subplots()</code>	2
<code>get_ipython().set_next_input('import numpy as np')</code>	3
<code>self.weights = np.random</code>	4
<code>ax=plt.subplot</code>	5
<code>n_samples, n_features = X.shape</code>	6
<code>self.weights = np.random.randn(n_features, 1)</code>	7
<code>self.bias = 0</code>	8
<code>z=np.dot(X,self.weights)</code>	9
<code>zipped = zip(X[:,0],self.weights[0])</code>	10
<code>def sigmoid(self, z):</code>	11
<code>zipped = zip(X[:,0],self.weights[0])</code>	12
<code>zipped = zip(X[:,0],self.weights[0])</code>	13
<code>zipped = zip(X[:,0],self.weights[0])</code>	14
<code>self.weights = self.lr</code>	15
<code>self.bias = self.lr</code>	16
<code>def predict(self):</code>	17
<code>return np.sum(weights) / n</code>	18

5.2 Aplicação: Predição de Preços de Imóveis

Category	Percentage
Very important	71%
Important	66%
Not important	59%
Don't know	1%
Very important	38%
Important	33%
Not important	3%
Don't know	2%
Very important	1%

```

scale = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

|

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

|

from sklearn.metrics import mean_squared_error, r2_score
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

```

5.3 Lições

- Implementação do zero revela a matemática.
- Convergência depende de learning rate e scaling.
- Scikit-learn é ~100x mais otimizado (usa LAPACK).

Diagrama: Regressão e Gradient Descent

Capítulo 6 — Regressão Linear com Scikit-Learn

6.1 API Padrão

```

from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

|

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

```

6.2 Regularização

Category	Percentage
1. The company's financial performance	35%
2. The company's reputation	25%
3. The company's products and services	20%
4. The company's customer service	15%
5. The company's social media presence	10%
6. The company's employee satisfaction	5%
7. The company's environmental impact	5%
8. The company's community involvement	5%

6.3 Diagnosticando o Modelo

Category	Percentage
Very important	10%
Important	25%
Not important	15%
Don't know	5%
Very important	35%
Important	45%
Not important	15%
Don't know	5%
Very important	55%
Important	35%
Not important	10%
Don't know	5%
Very important	40%
Important	45%
Not important	15%
Don't know	5%
Very important	30%
Important	40%
Not important	25%
Don't know	5%
Very important	50%
Important	40%
Not important	10%
Don't know	5%

Capítulo 7 — Regressão Logística — Detecção de Spam

7.1 0 Dataset

Usaremos o **SMS Spam Collection** (5574 mensagens).

Content Type	Percentage of Respondents
Text	100%
Images	95%
Videos	85%
GIFs	75%
Audio	60%
Animations	45%
Interactive	35%

7.2 Feature Engineering com TF-IDF

```

# Create a vectorizer object and fit it on the training data
vectorizer = TfidfVectorizer(max_features=1000, stop_words='english', min_df=2)
vectorizer.fit(train_data['text'])

# Transform the training data into a matrix of TF-IDF features
train_features = vectorizer.transform(train_data['text'])

# Transform the test data into a matrix of TF-IDF features
test_features = vectorizer.transform(test_data['text'])

```

7.3 Modelo

Task	Lines of Code
Linear Regression, Logistic Regression, Linear Support Vector	10
Linear Regression, Logistic Regression, Linear Support Vector	10
Linear Regression, Logistic Regression, Linear Support Vector	15
Linear Regression, Logistic Regression, Linear Support Vector	20
Linear Regression, Logistic Regression, Linear Support Vector	25
Linear Regression, Logistic Regression, Linear Support Vector	30
Linear Regression, Logistic Regression, Linear Support Vector	35
Linear Regression, Logistic Regression, Linear Support Vector	40
Linear Regression, Logistic Regression, Linear Support Vector	45
Linear Regression, Logistic Regression, Linear Support Vector	50
Linear Regression, Logistic Regression, Linear Support Vector	55
Linear Regression, Logistic Regression, Linear Support Vector	60
Linear Regression, Logistic Regression, Linear Support Vector	65
Linear Regression, Logistic Regression, Linear Support Vector	70
Linear Regression, Logistic Regression, Linear Support Vector	75
Linear Regression, Logistic Regression, Linear Support Vector	80
Linear Regression, Logistic Regression, Linear Support Vector	85
Linear Regression, Logistic Regression, Linear Support Vector	90
Linear Regression, Logistic Regression, Linear Support Vector	95
Linear Regression, Logistic Regression, Linear Support Vector	100

7.4 Threshold Tuning

Por padrão, $\text{threshold} = 0.5$. Em problemas desbalanceados, podemos ajustar:

```
from sklearn.metrics import confusion_matrix, recall_score  
  
y_pred = model.predict(X_test)  
cm = confusion_matrix(y_test, y_pred) # Confusion Matrix
```

```

recall = recall_score(y_test, y_pred, average='macro')
|
|
precision = precision_score(y_test, y_pred)
|
|
f1_score = f1_score(y_test, y_pred, average='macro')
|
|
recall_threshold = threshold_for_max_f1_score

```

7.5 Interpretação dos Coeficientes

```

feature_names = model.get_feature_names_out()
|
|
coeffs = pd.Series(model.coef_[0], index=feature_names)
|
|
coeffs.sort_values(ascending=False)
|
|
coeffs_int_names = coeffs.astype(int)
|
|
coeffs_int_names.sort_values(ascending=False)
|
|
coeffs_int_names

```

Capítulo 8 — Árvores de Decisão — Risco de Crédito

8.1 O Problema

Prever se um cliente **inadimplente** (default) com base em: - Idade - Renda - Histórico de crédito - Dívida atual - Score de crédito

8.2 Dataset Sintético

```

from sklearn.datasets import make_classification
|
|
X, y = make_classification(n_samples=1000, features=10, min_samples_per_class=10,
|
|
n_redundant=2, random_state=123)

```

8.3 Modelo

```

from sklearn.metrics import decision_function, roc_auc_score
|
|

```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.cross_validation import cross_val_score
from sklearn.cross_validation import cross_val_predict
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score

```

8.4 Análise de Importância

```

import matplotlib.pyplot as plt
import numpy as np
from sklearn import tree
from sklearn.metrics import feature_importances_
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score

```

Diagrama: Árvore de Decisão

Capítulo 9 — Random Forest — A Força do Ensemble

9.1 O Algoritmo

Random Forest (Breiman, 2001) combina **muitas árvores** treinadas em subamostras dos dados.

Por que funciona: - **Bagging:** Cada árvore vê ~63% dos dados (com reposição). - **Feature randomness:** Cada split considera apenas $\sqrt{p}$ features. - **Voting:** Predição = média (regressão) ou voto (classificação).

9.2 Código

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
|
|
| # RandomForestClassifier
| estimator = RandomForestClassifier(
|     n_estimators=100,
|     max_depth=5,
|     min_samples_split=10,
|     min_samples_leaf=5,
|     n_jobs=-1,
|     random_state=42,
| )
|
| # RandomForestRegressor
| estimator = RandomForestRegressor(
|     n_estimators=100,
```

9.3 Out-of-Bag (OOB) Score

```
oob_score = estimator.oob_score_
|
| # Out-of-Bag Score
| oob_score
```

9.4 Hyperparâmetros Críticos

- `n_estimators` : Mais árvores = mais estabilidade (diminishing returns).
- `max_depth` : Profundidade máxima (evita overfit).
- `min_samples_leaf` : Mínimo de amostras em folha.
- `max_features` : Features por split (sqrt para classificação, n/3 para regressão).

Capítulo 10 — Gradient Boosting (XGBoost, LightGBM)

10.1 O Paradigma

Boosting treina modelos **sequencialmente**, cada um corrigindo os erros do anterior.

11.2 Features de Filmes

```
movies = pd.DataFrame({
    'action': [0.9, 0.95, 0.9, 0.7],
    'romance': [0.2, 0.2, 0.2, 0.95],
    'sci-fi': [0.95, 0.95, 0.7, 0.1],
    'drama': [0.3, 0.5, 0.5, 0.9],
    'horror': [0.1, 0.1, 0.1, 0.1]
})
```

11.3 Modelo

```
from sklearn.neighbors import NearestNeighbor
from sklearn.preprocessing import StandardScaler

# Carregar o dataset
movies.dropna(inplace=True)

# Criar o scaler
scaler = StandardScaler()

# Aplicar o scaler
movies_scaled = scaler.fit_transform(movies)

# Criar o modelo
model = NearestNeighbor()

# Treinar o modelo
model.fit(movies_scaled)

# Para fazer a recomendação
para_filme = 'romance'
distancias = model.kneighbors(movies_scaled[para_filme])

# Para cada (movies_scaled[0][para_filme], recomendamos movies_scaled[indices][para_filme])
```

11.4 Similaridade Cosseno vs. Euclidiana

- **Euclidiana:** Distância absoluta. Boa para features em mesma escala.
 - **Cosseno:** Ângulo entre vetores. Ignora magnitude, bom para "perfil".
 - **Para recomendação de itens, cosseno quase sempre é melhor.**
-

Capítulo 12 — K-Means — Segmentação de Clientes

12.1 O Problema

Segmentar base de clientes de e-commerce em grupos acionáveis.

12.2 Dados Sintéticos RFM

```
import numpy as np
import pandas as pd
|
|
| random.seed(4)
|
| n = 100
|
| df = pd.DataFrame()
| recency = np.random.exponential(30, n) # Dias desde última compra
| frequency = np.random.poisson(5, n) # Compras/mês
| monetary = np.random.exponential(100, n) # valor médio
|
|
```

12.3 Modelo

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
|
| scaler = StandardScaler()
| data_scaled = scaler.fit_transform(df)
|
| n_clusters = 5
| model = KMeans(n_clusters=n_clusters, init='k-means++', random_state=42)
| model.fit(data_scaled)
| labels = model.labels_
| inertia = model.inertia_
| silhouette = silhouette_score(data_scaled, labels)
```

```

plt.figure(figsize=(10, 6))
plt.scatter(X, y)
plt.title('K-Means Clustering')
plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True)
plt.show()

```

12.4 K-Means++ e Múltiplas Inicializações

```

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42)
kmeans.fit(X)
cluster_labels = kmeans.predict(X)

```

12.5 Interpretação de Negócio

```

# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']
# Nome do cluster
cluster_names = ['Cluster 1', 'Cluster 2', 'Cluster 3', 'Cluster 4']

```

Diagrama: K-Means Clustering

Capítulo 13 — PCA — Compressão de Imagens

13.1 Visualizando o Efeito

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

digits = load_digits()
X, y = digits.data, digits.target
X = X.reshape(X.shape[0], -1)

pca = PCA(n_components=10)
pca.fit(X)
print(pca.explained_variance_ratio_)
plt.figure()
plt.xlabel('Component')
plt.ylabel('Explained variance ratio')
plt.bar(range(10), pca.explained_variance_ratio_, color='r', linestyle='-')
```

13.2 Reconstrução

```
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1)
plt.imshow(X[0].reshape(8, 8), cmap=gray)
plt.title('Original')
plt.subplot(2, 2, 2)
reduced = pca.fit_transform(X)
reconstructed = pca.inverse_transform(reduced)
plt.imshow(reconstructed[0].reshape(8, 8), cmap=gray)
plt.title('Reconstructed')
plt.subplot(2, 2, 3)
plt.imshow(X[0].reshape(8, 8), cmap=gray)
plt.title('Original')
plt.subplot(2, 2, 4)
plt.imshow(reconstructed[0].reshape(8, 8), cmap=gray)
plt.title('Reconstructed')
plt.tight_layout()
```

13.3 Taxa de Compressão

- **Original:** 64 features por imagem
- **Reduzido:** ~30 features (95% de variância)
- **Compressão:** ~2.1x sem perda perceptual significativa

13.4 Aplicações

- Pré-processamento para SVM/KNN (acelera treino).
- Visualização 2D/3D de datasets complexos.
- Remoção de ruído.
- Compressão de imagens.

Diagrama: PCA Dimensionality Reduction

Capítulo 14 — DBSCAN — Detecção de Fraude

14.1 Por que DBSCAN para Fraude?

- Não precisa especificar K.
- Identifica outliers como **ruído** (label -1).
- Encontra clusters de forma arbitrária.

14.2 Implementação

```
from sklearn.cluster import DBSCAN
import pandas as pd
import numpy as np

# Carregar o dataset de fraude
data = pd.read_csv('fraud_data.csv')

# Converter para numpy
X = data[['transaction_amount', 'time_elapsed', 'merchant_category']]

# Aplicar DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
clusters = db.fit_predict(X)

# Verificar os resultados
print(f"Clusters encontrados: {set(clusters).difference([-1])}")
print(f"Outliers (ruído): {len(clusters == -1)}")
```

```

scaled = scaler.fit_transform(X)

# DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=10, n_noise=1)
clusters = dbscan.fit_predict(scaled)

# Anomalias
anomalias = (clusters == -1).sum()
anomalias_percent = (anomalias / len(X_scaled)) * 100.0
print(f'Anomalias detectadas: {anomalias} ({anomalias_percent}%)')

# Comparar com Fraudes reais
fraudes_reais = y_scaled
fraudes_detectadas = (clusters == -1).sum()
print(f'Fraudes detectadas: {fraudes_detectadas} / {fraudes_reais}')

```

14.3 Tuning de ϵ e min_samples

Use o **k-distance plot**:

```

# k-distance plot
neighbors = nearest_neighbors(n_neighbors=5)
distances, _ = neighbors.kneighbors(scaled)

# Plot
plt.figure()
plt.plot(distances)
plt.xlabel('Pontos ordenados')
plt.ylabel('Distância até o vizinho')
plt.show()

```

O "cotovelo" no gráfico indica o bom valor de ϵ .

Capítulo 15 — Isolation Forest — Outliers em Séries Temporais

15.1 O Algoritmo

Isolation Forest (Liu et al., 2008) isola anomalias por meio de **partições aleatórias**. Anomalias são isoladas em **menos splits**.

15.2 Implementação

```
from sklearn.ensemble import IsolationForest
import numpy as np

# Séries temporais com 1000 pontos
X = np.random.randn(1000, 2)

# Parâmetros do modelo
n_estimators = 100
contaminated = 0.05

# Treinamento do modelo
iso_forest = IsolationForest(n_estimators=n_estimators,
                             contaminated=contaminated,
                             random_state=42)
iso_forest.fit(X)

# Predição de anomalias
anomalias = iso_forest.predict(X)

# Contagem de anomalias
n_anomalias = np.sum(anomalias == -1)
```

15.3 Vantagens

- Tempo linear: $O(n)$.
- Não usa distância (bom em alta dimensão).
- Funciona bem para séries temporais.

15.4 Caso de Uso Real: API Monitoring

```
from sklearn.ensemble import IsolationForest
import numpy as np

# Exemplo de dados de latência
latencies = np.array([10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, 110, 115, 120, 125, 130, 135, 140, 145, 150, 155, 160, 165, 170, 175, 180, 185, 190, 195, 200, 205, 210, 215, 220, 225, 230, 235, 240, 245, 250, 255, 260, 265, 270, 275, 280, 285, 290, 295, 300, 305, 310, 315, 320, 325, 330, 335, 340, 345, 350, 355, 360, 365, 370, 375, 380, 385, 390, 395, 400, 405, 410, 415, 420, 425, 430, 435, 440, 445, 450, 455, 460, 465, 470, 475, 480, 485, 490, 495, 500, 505, 510, 515, 520, 525, 530, 535, 540, 545, 550, 555, 560, 565, 570, 575, 580, 585, 590, 595, 600, 605, 610, 615, 620, 625, 630, 635, 640, 645, 650, 655, 660, 665, 670, 675, 680, 685, 690, 695, 700, 705, 710, 715, 720, 725, 730, 735, 740, 745, 750, 755, 760, 765, 770, 775, 780, 785, 790, 795, 800, 805, 810, 815, 820, 825, 830, 835, 840, 845, 850, 855, 860, 865, 870, 875, 880, 885, 890, 895, 900, 905, 910, 915, 920, 925, 930, 935, 940, 945, 950, 955, 960, 965, 970, 975, 980, 985, 990, 995, 1000])

# Treinamento do modelo
iso_forest = IsolationForest(contaminated=0.05)
iso_forest.fit(latencies)
```


[illegible]

```
from sklearn.svm import SVC
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
import re
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

# Fetch 20 newsgroups dataset
news = fetch_20newsgroups(subset='train', categories=10)

# Create news data matrix
X, y = news.data, news.target

# Create TF-IDF vectorizer
tfidf = TfidfVectorizer(stop_words='english')

# Fit and transform the data
X = tfidf.fit_transform(X)

# Create SVC model
svm = SVC(kernel='linear', C=1)

# Fit the model
svm.fit(X, y)

# Score the model
score = svm.score(X, y)
```

16.3 Tuning de C e gamma

Age Group	Percentage
18-29	92%
30-49	85%
50-69	78%
70+	72%
18-29	92%
30-49	85%
50-69	78%
70+	72%

Capítulo 17 — Naive Bayes — Filtro de Spam (Clássico)

17.1 O Algoritmo

Aplica o **Teorema de Bayes** com a suposição (ingênu) de independência entre features:

$$P(c|x) = \frac{P(x|c) P(c)}{P(x)}$$

17.2 Variantes

- **MultinomialNB:** Para contagens (ex: frequência de palavras).
- **BernoulliNB:** Para features binárias (palavra presente/ausente).
- **GaussianNB:** Para features contínuas (assume normalidade).

17.3 Código

[illegible]

```
|
|
|
|
```

17.4 Vantagens e Limitações

Vantagens: Treino ultra-rápido - Funciona bem com poucos dados - Interpretabilidade direta - Bom baseline para texto

Limitações: Suposição de independência raramente é válida - Não captura interações entre features - Pode ser superado por modelos mais complexos

Capítulo 18 — Validação Cruzada e Hyperparameter Tuning

18.1 Cross-Validation

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold
|
|
|
|
|
|
|
|
|
|
|
|
|
```

18.2 Grid Search

```
from sklearn.model_selection import GridSearchCV
|
|
|
|
|
|
|
```

```

|
|
|
| gridsearch
| RandomForestClassifier(random_state=1)
| param_grid
| cv
| scoring=f1
| n_jobs=-1
| verbose=1
|
|
| grid.fit(X_train, y_train)
| print(grid.best_params_)
| print(grid.best_score_)

```

18.3 Random Search

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform
|
| param_dist = {}
| param_dist.update({'n_estimators': randint(50, 500)})
| param_dist.update({'max_depth': randint(4, 20)})
| param_dist.update({'min_samples_split': randint(2, 20)})
| param_dist.update({'learning_rate': uniform(0.01, 1)})
|
|
| random_search = RandomizedSearchCV(
|     grid,
|     param_dist,
|     n_iter=100,
|     cv=5,
|     scoring=f1,
|     n_jobs=-1,
|     random_state=1,
| )
| random_search.fit(X_train, y_train)

```

18.4 Optuna (Bayesian Optimization)

```

import optuna
|
| study = optuna.create_study(
|     direction='maximize',
|     sampler=optuna.samplers.TPESampler(),
| )
| objective_trial

```



```

|
| numeric_features = ['age', 'salary']
| categorical_features = ['sex', 'gender']
|
| preprocessor = ColumnTransformer(
|     numeric=StandardScaler(numeric_features),
|     categorical=OneHotEncoder(handle_unknown='ignore')(categorical_features)
| )
|
|
| pipeline = Pipeline(
|     (preprocessor, preprocessor),
|     (classifier, RandomForestClassifier)
| )

```

19.3 MLflow — Tracking de Experimentos

```

import mlflow
import mlflow.sklearn

mlflow.set_experiment('customer-churn')

with mlflow.start_run():
    # Hyperparameters
    n_estimators = 100
    max_depth = 10
    mlflow.log_param('n_estimators', n_estimators)
    mlflow.log_param('max_depth', max_depth)
    |
    # Model
    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth)
    model.fit(X_train, y_train)
    |
    # Metrics
    y_pred = model.predict(X_test)
    f1_score = sklearn.metrics.f1_score(y_test, y_pred)
    mlflow.log_metric('f1_score', f1_score)
    mlflow.log_metric('auc', roc_auc_score(y_test, model.predict_proba(X_test)))
    |
    # Model
    mlflow.sklearn.log_model(model, 'model')
    |
    # Artifact
    mlflow.log_artifact('confusion_matrix.png')

```

Capítulo 20 — Deploy: Do Notebook à API

20.1 Salvando o Modelo

```
import pickle
from pathlib import Path
import joblib

model = joblib.dump(model, 'model.pkl')
```

20.2 FastAPI

```
from fastapi import FastAPI
from typing import List
import uvicorn

app = FastAPI()
model = joblib.load('model.pkl')

@app.post('/predict')
def predict(features: List[float]) -> float:
    """Return prediction for given features"""
    prediction = model.predict([features])
    return prediction[0]
```

```
python3 -m uvicorn main:app --host 0.0.0.0
```

20.3 Docker

```
from pydantic import BaseModel
from typing import List

class Input(BaseModel):
    """Input data"""
    data: List[int]

def main():
    """Main function"""
    # Install requirements
    # ...

if __name__ == '__main__':
    main()

python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
```

20.4 Batch Inference

```
import pandas as pd
import joblib

def main():
    """Main function"""
    # Load model
    model = joblib.load('model.pkl')

    # Load data
    new_data = pd.read_parquet('data/new_data.parquet')

    # Predict
    predictions = model.predict(new_data)

    # Save predictions
    new_predictions = predictions

    # ...
```

Capítulo 21 — Monitoramento em Produção (Drift)

21.1 Data Drift

A distribuição dos dados de entrada muda ao longo do tempo.

22.2 Stack Moderna

Camada	Ferramentas
Orquestração	Airflow, Prefect, Dagster
Feature Store	Feast, Tecton
Tracking	MLflow, Weights & Biases, Neptune
Model Serving	BentoML, Seldon, Triton, KServe
Monitoring	Evidently, Arize, Whylabs
CI/CD	GitHub Actions, GitLab CI
Infra	Kubernetes, Docker, Terraform

22.3 CI/CD para ML

cd \$GITHUB_WORKSPACE/bratn-vol
name: Train Model
on: push
paths: [data, src]
jobs:
build:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v2
- name: Setup Node
uses: actions/setup-node@v2
with:
node-version: 12
- name: Install deps
run: npm install
- name: Train
run: python src/train.py
- name: Validate
run: python src/validate.py
- name: Test
if: success()

Modelos de Machine Learning

Modelos de Machine Learning para o mundo real

22.4 Integração com Nexus Affil'IA'te

No ecossistema Nexus, **MLOps** é gerenciado pelo **SHO (Sistema Híbrido de Orquestração)**: - **Nível S2 (Supervisionado)**: Modelo treina, humano revisa amostral. - **Nível S3 (Autônomo)**: Retraining automático com guardrails. - **S4 (Federado)**: Modelos sincronizam entre nós federados.

Capítulo 23 — Algoritmos para Time Series

23.1 Especificidades

- **Ordem temporal**: Não pode shuffle.
- **Sazonalidade**: Padrões cíclicos (dia, semana, ano).
- **Autocorrelação**: Observações dependentes.
- **Não-estacionariedade**: Média/variância mudam.

23.2 Modelos Clássicos

Modelos de Machine Learning para o mundo real

Modelos de Machine Learning para o mundo real

|

Modelos de Machine Learning para o mundo real

Modelos de Machine Learning para o mundo real

Modelos de Machine Learning para o mundo real

Modelos de Machine Learning para o mundo real

|

Modelos de Machine Learning para o mundo real

Modelos de Machine Learning para o mundo real

23.3 Prophet (Facebook)

Category	Percentage
Not at all	10%
Not much	20%
A fair amount	40%
A great deal	30%

23.4 Deep Learning para Séries

- **LSTM/GRU:** Captura dependências de longo prazo.
- **Transformer-based:** Informer, Autoformer, PatchTST.
- **N-BEATS:** Especializado em forecasting.

Capítulo 24 — Algoritmos para NLP Clássico

24.1 Pipeline Clássico

Age Group	Percentage
18-29	95%
30-49	94%
50-69	88%
70+	87%
18-29	86%
30-49	85%
50-69	84%
70+	83%
18-29	82%
30-49	81%
50-69	80%
70+	79%
18-29	78%
30-49	77%
50-69	76%
70+	75%
18-29	74%
30-49	73%
50-69	72%
70+	71%
18-29	70%
30-49	69%
50-69	68%
70+	67%
18-29	66%
30-49	65%
50-69	64%
70+	63%
18-29	62%
30-49	61%
50-69	60%
70+	59%
18-29	58%
30-49	57%
50-69	56%
70+	55%
18-29	54%
30-49	53%
50-69	52%
70+	51%
18-29	50%
30-49	49%
50-69	48%
70+	47%
18-29	46%
30-49	45%
50-69	44%
70+	43%
18-29	42%
30-49	41%
50-69	40%
70+	39%
18-29	38%
30-49	37%
50-69	36%
70+	35%
18-29	34%
30-49	33%
50-69	32%
70+	31%
18-29	30%
30-49	29%
50-69	28%
70+	27%
18-29	26%
30-49	25%
50-69	24%
70+	23%
18-29	22%
30-49	21%
50-69	20%
70+	19%
18-29	18%
30-49	17%
50-69	16%
70+	15%
18-29	14%
30-49	13%
50-69	12%
70+	11%
18-29	10%
30-49	9%
50-69	8%
70+	7%
18-29	6%
30-49	5%
50-69	4%
70+	3%
18-29	2%
30-49	1%
50-69	0%
70+	0%

24.2 Técnicas Clássicas

- **Bag-of-Words (BoW):** Contagem de palavras.
- **TF-IDF:** Frequência ponderada por inverso da frequência em documentos.
- **Word2Vec/GloVe:** Embeddings pré-treinados.
- **Topic Modeling:** LDA, NMF.

24.3 Quando Usar vs. Transformers

Critério	Clássico	Transformer
Dataset pequeno		(overfit)
Velocidade		
Interpretabilidade		
Performance em texto longo		
Estado da arte		

Capítulo 25 — Quando NÃO Usar ML

25.1 As 7 Mortes do Projeto ML

1. **Regras claras funcionam:** Se dá para escrever `if/else`, faça.
2. **Dados insuficientes:** <100 amostras por classe.
3. **Causalidade confundida com correlação:** ML não descobre causalidade.
4. **Stakeholders não entendem o modelo:** Problema de gestão.
5. **Custo de erro é catastrófico:** Medicina crítica, aviação.
6. **Mudança de conceito rápida:** Modelo precisa retreinar a cada hora.
7. **Métrica de negócio vs. métrica de ML:** Foco errado.

25.2 O Teste "É ML?"

Faça estas perguntas: - O problema é uma **otimização** ou uma **regressão/classificação**? - Há **dados históricos** suficientes? - O **valor** de melhorar a baseline justifica o investimento? - A métrica é **mensurável** e **disponível**?

Se 3+ respostas forem sim, considere ML.

25.3 A Regra de Ouro

"Comece com o modelo mais simples (baseline). Só escale de complexidade se a métrica exigir."

Linha de base: **Regressão Logística** ou **Árvore de Decisão**. Se 80% do valor sai dela, pare aí.

Glossário

Termo	Definição
Bagging	Treinar modelos em subamostras (Random Forest).
Boosting	Treinar modelos sequencialmente corrigindo erros.
Cross-Validation	Avaliação usando múltiplos folds.
Drift	Mudança na distribuição dos dados ao longo do tempo.
Ensemble	Combinação de modelos.
Hyperparameter	Configuração externa ao modelo.
MLOps	Operações de ML em produção.
Pipeline	Sequência encadeada de transformações.
TF-IDF	Term Frequency-Inverse Document Frequency.

Termo	Definição
Train/Test Split	Divisão do dataset em treino e teste.

Referências

1. **Géron, A.** (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly.
2. **Hastie, T., Tibshirani, R., Friedman, J.** (2009). The Elements of Statistical Learning. Springer.
3. **Chen, T., Guestrin, C.** (2016). XGBoost: A Scalable Tree Boosting System. KDD.
4. **Ke, G. et al.** (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS.
5. **Prokhorenkova, L. et al.** (2018). CatBoost: Unbiased Boosting with Categorical Features. NeurIPS.
6. **Breiman, L.** (2001). Random Forests. Machine Learning.
7. **Liu, F. et al.** (2008). Isolation Forest. ICDM.
8. **Ester, M. et al.** (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. KDD.
9. **scikit-learn Documentation** (2024). <https://scikit-learn.org>.
10. **MLflow Documentation** (2024). <https://mlflow.org>.

Fim do Volume 03

"O melhor modelo é o que resolve o problema. Não o mais complexo, não o mais novo — o que resolve." — Nexus Affil'IA'te MMN_IA

Imagem de Encerramento